EXPERIMENTAL RESEARCH WHITEPAPER

MicroWorld: An Explicit Semantic Runtime for Deterministic and Auditable AI

Architecture, Evaluation, and Research Results — v2.0-alpha

AUTHOR
Arseniy Abramidze

PUBLICATION YEAR
2026

REPOSITORY
github.com/zowelqqee/microworld

LICENSE
Apache License 2.0

DOCUMENT STATUS
Experimental Research Whitepaper

BUILD COMMIT
e43984204c6c

COMPACT ABSTRACT

MicroWorld is an experimental, bounded semantic runtime that separates explicit memory, deterministic planning, transient dialogue state, safety policy, and controlled language rendering. It is designed to answer only from admitted support, to return an explicit negative decision for contradiction, and to audit unsupported, volatile, private, or ambiguous requests. Current repository evidence demonstrates local deterministic execution over checked-in artifacts, a 1,000-case controlled speech/reasoning stress suite, explicit dialogue-reference resolution, a proposal-only knowledge-acquisition path, and an offline iPhone demonstration. These results are workload-specific: they do not establish open-domain factual coverage, general natural-language reasoning, or superiority over language models.

Prepared from repository code, documentation, tests, and saved local artifacts. This paper reports implementation evidence, not peer review or independent replication.

Contents

1. Abstract
2. Introduction
3. Research question and hypothesis
4. Design principles
5. System architecture
6. Explicit semantic memory
7. Deterministic reasoning
8. Dialogue context and reference resolution
9. Controlled language generation
10. Knowledge acquisition and Knowledge Pump
11. Creative mode and architecture transfer
12. Local and on-device execution
13. Evaluation methodology
14. Results
15. Limitations
16. Relationship to other approaches
17. Future work
18. Conclusion
19. Reproducibility appendix
20. Author and license
21. Repository Evidence and References

1. Abstract

MicroWorld investigates whether useful bounded AI behavior can be assembled from explicit semantic entities, typed relations, deterministic planners, transient dialogue state, support policy, and a separate language layer. The project does not present this architecture as a replacement for modern language models. It is an experimental alternative for settings where a system should expose what it used, preserve an inspectable decision path, and decline to turn weak support into fluent assertion.

The runtime treats text as an interface to semantic structures rather than as the sole substrate of computation. It maintains distinct artifacts for accepted memory, overlays, proposals, snapshots, weak context, community-derived behavioral patterns, and dialogue state. A request is routed, parsed into a controlled semantic form, executed against the appropriate local structures, evaluated by support and temporal policy, converted into a semantic speech plan, and rendered into bounded English. An answer is therefore intended to follow admitted support; a contradiction may yield *no*; and an unsupported, current-sensitive, private, or ambiguous request is routed to *audit*.[2][7]

Repository artifacts show a deterministic speech/reasoning stress suite of 1,000 controlled prompts with 1,000 passes, 171 expected honest gaps handled as gaps, and zero recorded debug-like outputs, repetitive outputs, or decision mismatches. On that saved local CPU workload, latency was 8.05 ms p50, 29.47 ms p95, and 38.90 ms p99.[9] The repository also documents a dialogue benchmark of 21 sessions and a native SwiftUI iPhone demonstration that embeds CPython and executes the real runtime offline.[5][15]

These measurements are deliberately narrow. The stress suite is a deterministic suite over known categories, not 1,000 independent open-domain facts. The optional live-search path has a saved, weak WebQuestions-style result; device latency fields remain unfilled; and the creative system is generated text rather than factual support. The present evidence supports an implementation claim: within bounded explicit-memory workloads, MicroWorld can make local, inspectable, deterministic decisions and controlled failures. It does not establish general intelligence, broad open-domain competence, or independent reproducibility.

2. Introduction

Factual memory, inference, dialogue state, language generation, and safety are often experienced as one conversational surface. They need not, however, be one computational object. MicroWorld explores an engineering decomposition in which a system can keep a record of semantic support separate from the wording used to express it, and a record of dialogue references separate from durable factual memory. The intended benefit is not a claim that opaque next-token systems are invalid; it is the ability to inspect a narrower runtime's inputs, policies, intermediate plans, and failure decisions.

This choice matters most when plausible wording is not sufficient evidence. A natural-language response can sound coherent even when its relation direction, temporal scope, source, or referent is wrong. MicroWorld

therefore structures the answer path around entities, typed relations, mechanism roles, support kinds, source-qualified snapshots, and policy metadata. The renderer operates after a semantic answer decision, not before it. In the current implementation, the language layer may improve phrasing and style but is not permitted to make a factual claim true merely because it can express one fluently.[3][6]

The project is intentionally bounded. Its local artifacts determine its factual coverage; its parser recognizes controlled forms; and its safety model prefers an audit to a guess. This makes MicroWorld a useful research vehicle for explicit support, controlled failure, and artifact-level trust boundaries—not a general-purpose substitute for frontier language models.

3. Research question and hypothesis

Repository research question.
Can useful inference, memory growth, dialogue, controlled language generation, and trust learning be built from explicit semantic entities, typed relations, mechanism roles, safety policy, and deterministic planners instead of hidden model weights?[1]

The central hypothesis is architectural: bounded AI behavior can be useful when it combines explicit semantic entities; typed relations; deterministic planners; separate dialogue state; support and safety gates; controlled language generation; and inspectable memory artifacts. This is not a hypothesis that any one element is unprecedented, nor that every task should be solved this way. It is a hypothesis about the practical value of the combination and of maintaining the boundaries between its components.

The current implementation demonstrates portions of that hypothesis: supported relation and profile answers, deterministic audit routing, explicit dialogue-reference traces, separate speech evaluation, local execution, and proposal-oriented acquisition artifacts. Future goals—broader coverage, stronger extraction, QID-native identity, richer dialogue grammar, human evaluation, and stronger open-domain evaluation—remain plans rather than demonstrated outcomes.[8]

4. Design principles

Semantics before surface

Entities, typed relations, mechanism roles, support state, and references are the runtime substrate. Text is the input and output interface. A graph may store or traverse some of these structures, but MicroWorld does not define its core abstraction as a graph database.

Explicit, separated artifacts

Accepted memory, overlays, proposals, snapshots, weak context, community patterns, and dialogue state are kept apart so that a useful association cannot silently become durable factual support.

Deterministic planning

The controlled path uses explicit parsers, planners, indexes, gates, and renderers. The repository's dialogue layer uses type gates, integer salience, and margin rules rather than a latent chat-history selection.

Auditable failure

`audit` is a first-class result for missing, unsafe, volatile, or ambiguous support. It is a boundary decision, not an invitation to improvise.

Local-first operation

The demonstrated default path is local and can run without per-answer external model API calls. Optional live search is a separate volatile path rather than hidden memory.

Conservative support

Weak context, community patterns, and dialogue state can assist selection or explanation but cannot themselves establish a fact.

```
supported semantic claim present -> answer
explicit contradiction          -> no
weak, volatile, or current gap   -> audit
unknown or unsupported form     -> audit
```

The contract describes the intended decision discipline, not a proof of universal correctness. Its effectiveness is constrained by parser coverage, relation extraction quality, the completeness of local artifacts, and the accuracy of the policy classification.

5. System architecture

The runtime flow below consolidates the repository architecture into a print-safe diagram. The assistant surface routes the request; the semantic path produces a plan and decision; the language path realizes only an already-admitted plan. Optional live search and creative generation are deliberately distinct routes with different trust conditions.[2]

```
Text
-> Semantic Structures
-> Semantic Dialogue Context
-> Semantic Planning
-> Deterministic Execution
-> Safety and Support Gate
-> Semantic Speech Plan
-> Controlled Renderer
-> Answer / No / Audit
```

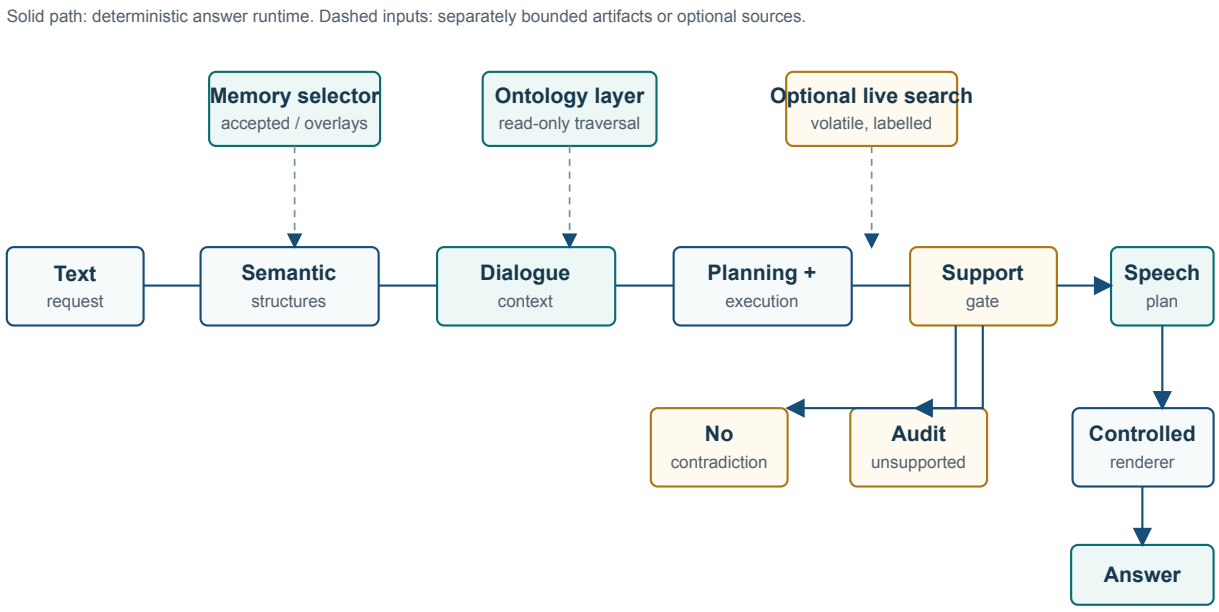

Figure 1. Runtime architecture synthesized from the documented module boundaries. The diagram omits no required trust boundary: live search is optional and labelled; community context is not a factual-support input.

Responsibility	Repository module(s)	Boundary
Assistant surface and router	`worldpgt/assistant_surface/`	Chooses factual, safety, web-search, or creative route; attaches traces.
Semantic memory selector	`worldpgt/assistant_surface/context_selector.py` ; `worldpgt/knowledge/` ; overlay artifacts under `worldpgt/experiments/`	Selects explicit local structures; proposals remain separate.
Parser, planner, and executor	`worldpgt/entity_qa/semantic_question_parser.py` , `entity_answer_planner.py` , `query_engine/` , `multihop_qa/` , `cross_page_qa/`	Transforms controlled forms into lookup, path, count, comparison, traversal, or synthesis plans.
Dialogue state and reference resolver	`worldpgt/dialogue/`	Transient, replayable selection state; no factual writes.
Safety and temporal policy	`worldpgt/assistant_surface/` , `worldpgt/knowledge/` , `relation_extraction_v2/relation_policy.py`	Blocks unsupported, private, volatile, inverted, or ambiguous claims.
Speech planner, phrase graph, and renderer	`entity_qa/semantic_speech_planner.py` , `symbolic_text_generator.py` , `cognition/phrase_graph.py`	Realizes supported plans; does not supply factual support.
Optional live search	`worldpgt/web_search/`	Volatile source-labelled path; not accepted memory.
Creative mode	`worldpgt/cognition/creative_generator.py` ; `poetry_lab/`	Generated, labelled, non-factual output under a novelty gate.

6. Explicit semantic memory

MicroWorld uses “memory” as a family of different semantic artifacts rather than a single truth store. The separation is central: a source, candidate, context link, or session reference may be useful without being eligible to support a durable factual claim.[2][7]

Layer	Meaning in the current runtime	Support status
Accepted memory	Trusted explicit semantic memory artifact.	Admitted factual support under policy.
Accepted wiki overlay	Isolated overlay of explicit wiki-derived semantics.	Separate from accepted memory; used only under its runtime contract.
Promoted overlay	Output of explicit review/promotion process.	Still a separate artifact; it does not overwrite accepted memory.
Proposal / pump dry-run overlay	Candidate extraction and runtime experiment artifact.	Proposal-only; not silently trusted or promoted.
Snapshot	Source-qualified, dated input for a bounded claim.	Must retain source and temporal qualification.
Weak and community context	Associations, speech habits, or behavioral patterns.	Cannot establish facts.
Dialogue state	Session pointers, roles, references, and turn records.	May identify a referent; cannot create factual support.

Inspectability here means more than choosing JSON as a storage format. A semantic runtime needs relations, source status, trust boundaries, and policies that survive serialization and inspection. A graph database can be a useful implementation for relationships and traversal, but it does not by itself define the planning, support, temporal, dialogue, or speech contracts. Conversely, a JSON overlay or index can participate in a semantic runtime if it preserves those contracts.

7. Deterministic reasoning

MicroWorld does not claim unrestricted natural-language reasoning. Its current parser and planner implement controlled semantic behaviors: direct relation lookup, inverse relation lookup, entity synthesis, relation traversal, classification, filtering, counting, comparison, connection paths, ontology `is_a` traversal, contradiction handling, and audit outcomes for gaps or unsupported forms.[3]

Behavior	Representative repository example	Decision boundary
Direct relation	“Who founded SpaceX?” → founded_by relation.	Requires explicit supported relation.
Inverse relation	“What did Elon Musk found?”	Direction is checked; inverted unsupported assertions audit.
Profile / synthesis	“Tell me about SpaceX.”	Bundles definition and selected supported roles.
Connection path	“How is Starlink connected to Falcon 9?”	Renders an explicit path through SpaceX.
Classification / contradiction	“Is SpaceX a person?” → no.	Requires explicit type evidence that contradicts the claim.
Count / comparison	“How many companies did Elon Musk found?”; comparative founder questions.	Limited to parser/planner-recognized relations and entities.
Mechanism-oriented query	“How does Starlink work?”	May produce a supported partial answer with an explicit missing-mechanism gap.

The executor is deterministic in the sense relevant to this paper: it follows explicit code and artifact data, rather than using an unobserved generative model to fill gaps. That does not make every upstream fact correct. Extraction mistakes, aliases, missing relations, and parser errors remain possible; deterministic behavior makes these failure modes easier to isolate and test.

8. Dialogue context and reference resolution

Dialogue state reduces a follow-up to a resolved single-turn semantic question. The v2 design specifies `DialogueState` as the session memory, `TurnRecord` as its mutation input, and `resolve_question(question, state, index)` as a pure step before semantic parsing. Candidate selection uses type gates, integer salience, a required margin, and an explanation trace. When a required reference cannot be resolved confidently, the request audits rather than selecting the top candidate by default.[5]

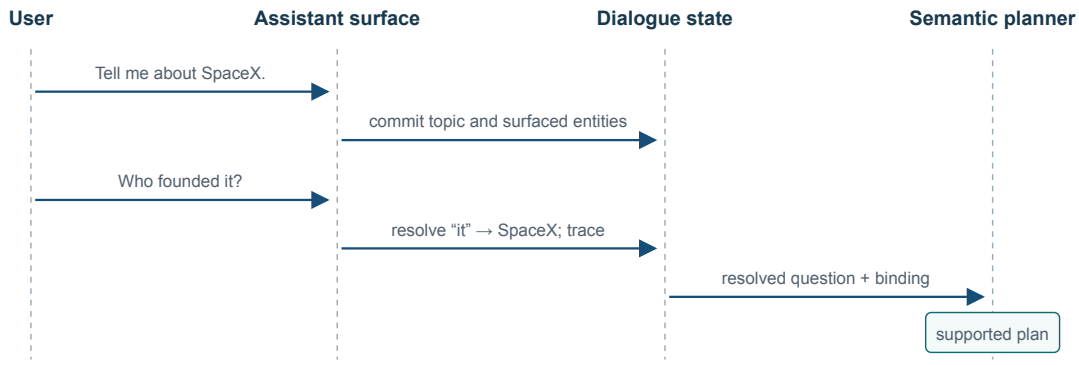

Figure 2. Reference resolution selects a previously known entity and passes a bound question to planning. It does not create a relation such as `founded_by`.

The documented sequence gives a concrete example: after “Tell me about SpaceX,” the reference “it” resolves to SpaceX. After an explicit topic shift to Elon Musk, “What else did he found?” resolves to Elon Musk and excludes the already surfaced SpaceX founding relation. A later “What about Tesla?” is an explicit topic shift, after which “Who founded it?” resolves to Tesla. The resolver also demonstrates the inverse boundary: a multi-founder OpenAI response followed by “What did he found?” audits when several equally salient male candidates have no sufficient margin.[5]

9. Controlled language generation

MicroWorld separates fact selection, reasoning, speech planning, phrase selection, rendering, and style control. The planner selects supported roles—such as definition, activity, purpose, relation, or mechanism—and the cognitive layer records an action plan such as `answer`, `answer_with_gap`, `audit`, or clarification. The symbolic renderer then produces English from this constrained plan. The architectural claim is therefore not "facts are generated," but "speech is generated over admitted facts."[6]

The phrase graph contains locally learned phrase fragments and transitions. Repository documentation describes deterministic, frequency-weighted seeded fragment selection; grammar-gated fragment eligibility; fact bundling that combines consecutive capabilities; and relation fusion that prevents a forward and inverse representation of the same founding fact from becoming two awkward sentences. The surface-selection layer checks for debug-like wording and repetition, while the benchmark records those checks separately from factual coverage.[6][9]

Layer	Character of behavior	What it must not do
Semantic fact selection	Explicit memory/planner result.	Infer unsupported facts from fluent language.
Reasoning trace and support guard	Rule-based, deterministic over current code and artifacts.	Promote a mechanism gap into a mechanism claim.
Speech plan	Structured roles and clause order.	Change the answer/no/audit decision.
Phrase graph	Local corpus-derived fragments with deterministic seeded selection.	Act as a factual source.
Renderer and style control	Rule-based/bounded realization, including brief and detailed forms.	Override support policy or reference identity.

This is a hybrid system in a narrow sense: local corpus statistics influence some phrase choices, while the decision path and eligibility gates remain explicit. That distinction matters. Learned surface fragments may make an answer less brittle; they do not add coverage or alter the trust class of a claim.

10. Knowledge acquisition and Knowledge Pump

The Knowledge Pump is a proposal-oriented acquisition pipeline. It reads or fetches candidate pages, extracts claims using deterministic patterns and optionally spaCy, applies relation and precision policy, runs fact

checks and regression gates, and writes a dry-run overlay. Promotion is a separate review artifact; accepted memory is not silently mutated.[4][10]

Current saved pump snapshot	Value	Interpretation
Batches completed	275	Acquisition progress, not a measure of broad truth coverage.
Fetched / successful pages	22,620 / 7,120	Operational extraction statistics.
Extraction v2 candidates / safe deltas	4,496 / 4,438	Candidate and filtered-artifact counts.
Answerable fact delta	2,836	Proposal/runtime delta; not accepted memory.
Dedicated pump fact QA	570 facts; 1,200 prompts; 0 wrong; 0 unsupported answers	Controlled QA artifact with 11 planner gaps; it does not yet cover the full 2,836-fact delta.

The frontier is yield-ranked from prior batches, can grow dynamically from newly fetched pages, and can receive structured gap signals from audit outcomes. These mechanisms optimize an acquisition process; they do not justify treating raw extraction output as trusted truth. The source artifact expressly records `trusted_memory_modified: false`, `accepted_overlay_modified: false`, and `safe_for_general_runtime: false` for the current pump summary.[10]

11. Creative mode and architecture-transfer experiment

`poetry_lab/` is a separate architecture-transfer experiment: it asks whether the structural decomposition used in QA can produce creative writing when the knowledge source changes from factual artifacts to a literary corpus. It preserves typed concepts, spreading activation, a frequency phrase graph, deterministic seeded picks, discourse state, explicit planning artifacts, and JSON boundaries; it replaces factual ingestion and inverts the support gate.[13]

```
Factual mode:
allow output only when claims are grounded.

Creative mode:
allow output only when generated text passes the novelty gate and does not recite
protected source fragments.
```

The creative path is not factual support. The main runtime labels it as generated and returns `supported=False` with `support_kind='creative_generated'`. A hard safety screen remains ahead of it, so private or current-sensitive prompts can still audit. The poetry experiment's novelty gate blocks output that reproduces a corpus 4-gram; it does not turn a literary fragment into a verified statement about the world.[11] [13]

The repository documents several controlled before/after evaluations. On a fixed Russian-corpus setup, the order-1-to-order-2 phrase-model change raised local grammaticality (attested generated three-word windows) from 0.19 to 0.79 while novelty remained 0.99. A discourse-salience selection experiment reports inter-line continuity rising from 0.13 to 0.23, alongside metric trade-offs and a documented correction to the meter gate. These are internal experiment measurements, not human literary evaluation and not a general claim about creative quality.[13]

Transfer later became bidirectional: the poetry documentation says description-mode fact bundling developed in the experiment was transferred back into production QA. This is evidence of shared architectural motifs, not evidence that the systems solve the same factual or artistic problem.

12. Local and on-device execution

The standalone runtime documents a local installation path with no model download, API key, database, or external service for the bundled demo. Its default use reads included local artifacts; optional live search must be explicitly configured. The speech stress artifact describes a local CPU workload with no GPU or model API at answer time.[14][9]

The iOS demonstration is native SwiftUI. It embeds CPython, stages the unmodified `worldpgt` package and a thin `mw_ios.py` adapter, and invokes the real `AnswerOrchestrator` rather than a precomputed answer table. The technical decision records a dependency-blocking check for the answer path and describes the intended offline network guard; the iOS README documents a physical iPhone 11 offline demo.[15][16]

> **Device measurement boundary.** The device benchmark explicitly separates Mac reference values from device values. It reports confirmed memory stages—approximately 42 MB for warm QA and approximately 400 MB after warming the slim creative artifact—but leaves device latency and launch fields blank for Instruments. This paper therefore does not report Mac timing as iPhone timing.[17]

13. Evaluation methodology

MicroWorld has several distinct evaluation artifacts. They should not be combined into a single performance claim because they measure different questions: controlled answer-surface stability, dialogue resolution, knowledge-pump QA, optional live-search behavior, creative structure, and local resource feasibility.

Evaluation	Purpose and workload	Saved/command evidence	What it does not measure
Speech/reasoning smoke	Fast contract check over 12 cases.	`benchmark_speech_quality_v1.py --suite smoke`	Open-domain factual coverage.
Large speech suite	50 controlled prompts across profile, relation, path, safety, gap, and style categories.	`speech_quality_large_20260709T111746Z.json`	Independent conversational quality or broad QA accuracy.
Stress speech suite	1,000 deterministic prompts over known categories; records latency and surface checks.	`speech_quality_stress_20260709T111906Z.json`	1,000 independent open-domain facts; general intelligence.
Dialogue benchmark	21 sessions, resolver traces and replay/ambiguity behavior.	`python3 -m worldpgt.benchmarks.dialogue_benchmark`	Broad natural dialogue understanding.
Pump fact QA	1,200 prompts over 570 facts, adversarial and current-safety cases.	`pump_fact_qa_summary.json`	Full pump-delta coverage or independent source verification.
Live-search snapshot	250 WebQuestions-style questions with optional search.	Referenced in docs as `external_20260706T203034Z.json`	A strong open-domain result; the artifact is not present in this trimmed tree.
Creative transfer evaluations	Fixed prompt batteries and structural metrics such as novelty, local grammaticality, and continuity.	`poetry_lab/eval/` and README result tables	Human literary-quality evaluation.

Saved speech result	Large suite	Stress suite
Questions / passed	50 / 50	1,000 / 1,000
Quality rate	100.0%	100.0%
Expected honest gaps handled	9 / 9	171 / 171
Debug-like / repetitive / decision drift	0 / 0 / 0	0 / 0 / 0
Latency p50 / p95 / p99	3.39 / 26.98 / 27.73 ms	8.05 / 29.47 / 38.90 ms
Max latency	28.16 ms	123.53 ms

The quality rate is a suite-specific result defined by the benchmark's checks; it is not a general answer-quality score. "Honest gaps" counts expected gap cases classified without the benchmark's designated error conditions.

14. Results

1,000 / 1,000
Passed in the saved deterministic stress suite.

8.05 ms
Stress-suite p50 latency on the saved local CPU artifact.

21 / 21
Dialogue benchmark sessions passed in the documented trimmed-runtime validation.

0 wrong
Saved pump fact-QA artifact across 1,200 controlled prompts; 11 planner gaps remain.

The current implementation demonstrates that explicit semantic support can drive deterministic answer decisions in its evaluated bounded workload; unsupported requests can route to audit; dialogue references can be selected from explicit state; and speech rendering can be evaluated separately from factual coverage. It also demonstrates a local execution path and documents a physical iPhone 11 offline demonstration. Finally, the creative experiment shows that selected mechanisms—activation, phrase fragments, planning artifacts, discourse state, and a gate—can transfer across factual and creative domains when their trust conditions are changed explicitly.[5][9][13]

What the result does *not* demonstrate is equally important: that MicroWorld knows an open world, that a 100% controlled-suite result generalizes outside the suite, that its optional live search is strong, that it outperforms language models, or that a device's latency has been characterized. Those claims are outside the supplied evidence.

15. Limitations

- **Bounded coverage.** The runtime depends on explicit local entities, relations, snapshots, and extraction quality. It audits many unknown forms rather than resolving them.
- **Limited language understanding.** The parser and reference grammar are controlled; unfamiliar paraphrases, ambiguous language, and novel constructions can fall outside supported forms.
- **Artifact dependence.** Alias/surface identity remains imperfect; the current ontology layer is read-only and is not a QID-native identity system.
- **Deterministic benchmark construction.** The 1,000-question stress suite measures stability over known categories, not broad factual recall or independent open-domain data.
- **No broad comparative evaluation.** The repository does not establish a fair comparison with current frontier language models, retrieval systems, or knowledge-graph products.
- **Weak live-search evidence.** Documentation records 28.57% precision among answered rows in a 250-question saved WebQuestions-style snapshot, and the cited JSON is absent from this trimmed tree.
- **Extraction and pump gaps.** The fact-QA artifact covers 570 facts while the current answerable-fact delta is 2,836; promotion remains deliberately incomplete and explicit.
- **Renderer brittleness.** Rule and phrase-fragment systems can make awkward or uneven text; surface fluency does not increase factual coverage.
- **Creative limits.** Creative output is corpus-bound, internally measured, and not human-evaluated here. It can audit or yield no passage for poorly supported prompts.
- **Limited device study.** Physical iPhone operation and memory stages are documented, but device latency, launch time, installed bundle size, and repeated independent measurements remain pending.
- **Experimental status.** The repository does not claim peer review, independent reproduction, or production readiness.

16. Relationship to other approaches

This is conceptual positioning based on the implementation, not a literature review. No external academic bibliography is asserted here.

Approach	Conceptual relationship to MicroWorld
Large language models	MicroWorld separates semantic support, policy, dialogue state, and rendering rather than using a single next-token model for all of them. It does not claim to replace LLM breadth, fluency, or learned world knowledge.
Knowledge graphs	Graphs can be a storage/traversal technique for relations, but MicroWorld's claimed focus is the surrounding semantic, policy, dialogue, and language contracts.
Symbolic AI and expert systems	It shares explicit rules, representations, and bounded inference, while adding artifact-level overlays, support/temporal policy, controlled rendering, and current implementation-specific dialogue traces.
Retrieval-augmented generation	Its optional live-search route retrieves volatile text, but the route is kept distinct from admitted memory and is source-labelled. The project does not claim a general RAG solution.
Neuro-symbolic systems	Some local phrase and relation-label mechanisms use corpus-induced statistics or optional embeddings, but the documented answer path is predominantly explicit and deterministic. No claim is made about the broader field.
Semantic parsers and dialogue-state tracking	MicroWorld implements controlled semantic parsing and an explicit dialogue state with type gates, salience, and traceable reference bindings. These are bounded implementations, not general language understanding.

The specific implementation focus is the combination of explicit-memory layers, support policy, deterministic semantic planning, transient dialogue selection, controlled rendering, and proposal-only acquisition. The repository does not establish that individual components or the combination are novel across the research field.

17. Future work

Repository documentation identifies the following planned directions: broaden semantic coverage; add concrete mechanism evidence roles such as `uses` and `works_by`; tighten extraction precision before widening explanatory predicates; refresh fact QA to cover the current 2,836-item delta; develop QID-native identity; continue dialogue-v2 migration beyond shadow validation; connect community cognitive patterns to reasoning without making them factual support; improve live-search precision and source relevance; record repeated latency/stability environments; expand adversarial and human evaluation; and collect fuller device measurements.[4][8]

These are plans, not completed claims. The safest progression remains the project's existing discipline: make an artifact-level change, validate it against focused tests and a runtime path, preserve trust boundaries, and

report both gains and gaps.

18. Conclusion

MicroWorld is an experimental semantic AI runtime. Within bounded domains defined by explicit local artifacts, it is designed to make deterministic, inspectable answer, no, and audit decisions; to keep dialogue references separate from factual memory; and to render supported content through a controlled language layer. The saved repository evidence supports local millisecond-scale behavior for a controlled answer-surface workload, explicit gap handling, dialogue-state evaluation, proposal-only knowledge acquisition, and offline on-device execution.

It is not an open-domain replacement for frontier language models. Its practical value depends on the quality and coverage of its artifacts, parser, policy, and renderer. The research contribution under test is narrower: whether explicit semantic boundaries can make a useful class of AI behavior inspectable, correctable, and deliberately conservative when support is absent.

19. Reproducibility appendix

Standalone runtime

```
git clone https://github.com/zowelqqee/microworld.git
cd microworld/microworld-standalone
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install .
microworld "Who founded SpaceX?" --overlay pump-dry-run
microworld --overlay pump-dry-run --interactive
microworld-api --overlay pump-dry-run --port 8000
```

Benchmark and dialogue commands

```
python3 -m worldpgt.experiments.benchmark_speech_quality_v1 --suite large
python3 -m worldpgt.experiments.benchmark_speech_quality_v1 --suite stress
python3 -m pytest worldpgt/tests/test_benchmark_speech_quality_v1.py -q
python3 -m worldpgt.benchmarks.dialogue_benchmark
python3 -m pytest worldpgt/tests/test_dialogue_state.py \
  worldpgt/tests/test_reference_grammar.py \
  worldpgt/tests/test_salience.py \
  worldpgt/tests/test_resolver.py \
  worldpgt/tests/test_dialogue_benchmark_v1.py -q
```

Key artifact paths

```
worldpgt/experiments/benchmarks/speech_quality_large_20260709T111746Z.json
worldpgt/experiments/benchmarks/speech_quality_stress_20260709T111906Z.json
worldpgt/experiments/knowledge_pump_v1/pump_summary.json
worldpgt/experiments/knowledge_pump_v1/pump_fact_qa_v1/pump_fact_qa_summary.json
worldpgt/experiments/community_context_v1/reddit_community_summary.json
worldpgt/benchmarks/fixtures/dialogue_sessions_v1.json
ios_demo/DEVICE_BENCHMARK.md
poetry_lab/README.md
```

Repository structure

```
worldpgt/                runtime package
  assistant_surface/     routing, support checks, orchestration
  cognition/             traces, support guard, phrase graph, creative route
  dialogue/              replayable state and reference resolver
  entity_qa/             semantic parse, plan, synthesis, speech plan
  knowledge/             memory providers, temporal and ontology support
  knowledge_pump/        proposal-oriented acquisition and gates
  experiments/           checked-in local artifacts and commands
microworld-standalone/   self-contained installation surface
ios_demo/                native SwiftUI / embedded CPython demonstration
poetry_lab/              creative architecture-transfer experiment
```

Build metadata: document build commit `e43984204c6c` . To reproduce a different revision, replace this value with `git rev-parse --short=12 HEAD` from the repository root before publication.

20. Author and license

21. Repository Evidence and References

These numbered references point to repository evidence used in this whitepaper. They are implementation and artifact references, not external academic citations.

[1] `README.md` — project framing, research question, decision contract, high-level runtime path, and saved benchmark pointers.

[2] `docs/architecture.md` — module boundaries, memory layers, optional live search, community-context boundary, and current artifact map.

[3] `docs/semantic_runtime.md` — semantic-first runtime definition, supported query forms, examples, and controlled scope.

[4] `docs/knowledge_pump.md` — proposal-only acquisition design, current pump snapshot, QA caveats, and planned work.

[5] `docs/dialogue_context.md` ; `worldpgt/dialogue/state.py` ; `resolver.py` ; `reference_grammar.py` — explicit state, deterministic reference resolution, margin-based ambiguity handling, and benchmark result.

[6] `docs/language_renderer.md` ; `worldpgt/cognition/phrase_graph.py` ; `worldpgt/entity_qa/semantic_speech_planner.py` — speech-plan boundary, phrase fragments, rendering, bundling, and surface checks.

[7] `docs/safety_model.md` ; `worldpgt/assistant_surface/answer_orchestrator.py` — support/temporal gates, community and live-search boundaries, audit routing, and creative labelling.

[8] `docs/research_results.md` — preserved research framing, current limitations, and next-work priorities.

[9] `docs/benchmarks.md` ; `worldpgt/experiments/benchmark_speech_quality_v1.py` ; `worldpgt/experiments/benchmarks/speech_quality_large_20260709T111746Z.json` ; `speech_quality_stress_20260709T111906Z.json` — controlled speech/reasoning method, results, categories, and latency.

[10] `worldpgt/experiments/knowledge_pump_v1/pump_summary.json` ; `pump_fact_qa_v1/pump_fact_qa_summary.json` ; `worldpgt/knowledge_pump/yield_ranked_frontier.py` ; `safe_delta_merger.py` — acquisition counts, proposal safety flags, fact-QA results, frontier and merge mechanisms.

[11] `worldpgt/cognition/creative_generator.py` ; `worldpgt/assistant_surface/answer_orchestrator.py` — separate creative route, generated label, novelty gate trace, and non-factual support status.

[12] `worldpgt/query_engine/` ; `worldpgt/multihop_qa/` ; `worldpgt/cross_page_qa/` ; `worldpgt/entity_qa/` — query primitives, traversal, path planning, semantic question analysis, and answer planning.

[13] `poetry_lab/README.md` ; `poetry_lab/poemcore/{concept_graph.py,phrase_model.py,discourse.py,narrative.py,novelty.py}` ; `poetry_lab/eval/` — architecture transfer, inverted novelty gate, creative pipeline, and internal before/after metrics.

[14] `microworld-standalone/README.md` ; `requirements.txt` — standalone local installation, offline default, CLI/API commands, and included artifact boundaries.

[15] `ios_demo/README_IOS.md` — SwiftUI/CPython integration, offline-design claims, build instructions, and iPhone demonstration description.

[16] `ios_demo/TECHNICAL_DECISION.md` — rationale for embedding CPython, dependency-blocked answer-path test, reference-only Mac figures, and adapter boundary.

[17] `ios_demo/DEVICE_BENCHMARK.md` — separation of Mac and device measurements, confirmed memory stages, slim-artifact notes, and unfilled latency fields.

[18] `LICENSE` ; `NOTICE` ; `AUTHORS` — Apache License 2.0 and author/copyright notices.

[19] `worldpgt/tests/` — focused coverage for assistant surface, dialogue, parser, query engine, phrase graph, creative mode, pump, and benchmark contracts.